СОДЕРЖАНИЕ

ВВЕДЕНИЕ 7
1 Постановка задачи 8
2 Проектирование плагинной архитектуры 9
2.1 Бэкенд: плагинная система на основе ServiceLoader 9
2.2 Образ Docker 10
2.3 Фронтенд: динамическая подгрузка языков 11
3 Проектирование хранилища MinIO 13
3.1 Обоснование выбора MinIO 13
3.2 Структура объектного хранилища 14
3.3 Интеграция в архитектуру микросервисов 15
ЗАКЛЮЧЕНИЕ 17
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 18

ВВЕДЕНИЕ

В современном мире информационные технологии играют ведущую роль в развитии различных отраслей экономики и науки. Следовательно, спрос на IT-специалистов, включая разработчиков программного обеспечения, постоянно растет. В связи с этим, возникает необходимость в подготовке новых специалистов в области программирования. Образовательные информационные системы с алгоритмическими задачами по программированию становятся неотъемлемой частью этого процесса подготовки. Эти системы предоставляют студентам и начинающим программистам возможность развивать свои навыки, решая разнообразные задачи с различным уровнем сложности. Благодаря таким системам студенты могут улучшить свои навыки алгоритмизации, научиться эффективно решать задачи и практиковаться в программировании на различных языках. Более того, образовательные информационные системы обеспечивают доступ к обучающим материалам, а также предоставляют возможность соревноваться с другими пользователями, что мотивирует студентов к развитию и позволяет им оценить свой прогресс в сравнении с другими. Такие системы также часто предлагают системы автоматической проверки решений, что позволяет студентам получать обратную связь и исправлять ошибки, улучшая свои навыки программирования.

1 Постановка задачи

В рамках дальнейшего научного исследования планируется разработать информационную систему обучения и проверки навыков программирования «SupremeCode» с автоматизированной проверкой решений задач.

Для выполнения поставленной цели необходимо сосредоточиться на двух ключевых задачах: реализации механизма плагинизации для расширяемости и поддержки платформы, а также организации хранения файловых данных с использованием объектного хранилища MinIO [1], чтобы исключить хранение файлов в базе данных PostgreSQL.

В связи с определенными выше объектом и предметом исследования была поставлена цель работы, которая заключается в выборе и обосновании архитектурных решений, повышающих расширяемость и поддерживаемость платформы.

Для достижения поставленной цели в данном разделе отчёта рассматриваются следующие задачи:

– спроектировать плагинную архитектуру [2], позволяющую расширять платформу новыми языками программирования без модификации её ядра, что обеспечит долгосрочную поддержку и адаптивность системы;

– спроектировать механизм хранения файлов (включая исходные коды решений, тестовые наборы и отчёты) во внешнем объектном хранилище MinIO, исключив хранение бинарных данных в реляционной базе PostgreSQL в соответствии с принципами нормализации и эффективного управления ресурсами.

2 Проектирование плагинной архитектуры

Для обеспечения гибкости и расширяемости платформы была разработана плагинная архитектура, позволяющая поддерживать произвольные языки программирования без модификации ядра системы. Архитектура реализована как на стороне бэкенда (на языке Java/Kotlin), так и на фронтенде (на TypeScript с использованием React). Такой подход позволяет независимо добавлять новые языки, не затрагивая основной код платформы, и обеспечивает чёткое разделение ответственности между компонентами.

2.1 Бэкенд: плагинная система на основе ServiceLoader

На уровне бэкенда реализована модульная система, основанная на механизме java.util.ServiceLoader [3]. Для этого в состав SDK включены следующие ключевые интерфейсы:

– LanguageInfo — предоставляет метаданные о языке программирования, такие как уникальный идентификатор, отображаемое имя, а также параметры интеграции с редактором Monaco [4];

– LanguageTester — определяет контракт для выполнения тестирования пользовательского решения, включая указание Docker-образа, путей к исходному коду и отчёту о тестировании, а также метод преобразования отчёта в унифицированный результат;

– LanguageWeb — позволяет плагину обрабатывать HTTP-запросы напрямую, что может быть полезно для реализации специфичных веб-интерфейсов.

Для упрощения разработки плагинов в SDK [5] включён абстрактный класс Tester, реализующий типовые операции, связанные с изоляцией и запуском кода в контейнере:

– создание tar-архива с пользовательским решением;

– запуск Docker-контейнера с заданными ограничениями ресурсов;

– копирование исходного кода в контейнер;

– отслеживание логов выполнения;

– извлечение результатов тестирования из контейнера;

– корректное завершение и удаление контейнера.

Разработчик плагина обязан реализовать указанные интерфейсы и собрать свой проект в виде fat-JAR с применением shading-техник, чтобы избежать конфликтов зависимостей. Платформа автоматически загружает все JAR-файлы из предварительно сконфигурированной директории (plugins по умолчанию) с использованием изолированного URLClassLoader [6], что гарантирует независимость плагинов друг от друга и от основного приложения.

Также в состав SDK входят вспомогательные компоненты:

– DTO Result — унифицированное представление результата выполнения тестов;

– парсер отчётов JUnit в формате XML на основе JAXB, покрывающий около 90 % распространённых случаев генерации отчётов;

– набор unit-тестов для проверки корректности загрузки и инициализации плагинов.

2.2 Образ Docker

Каждый плагин определяет собственный Docker-образ, который должен быть заранее подготовлен разработчиком языкового пакета. В образ включаются:

– необходимые системные зависимости и runtime-окружение;

– библиотеки, требуемые для выполнения решений;

– предварительно настроенный тестовый фреймворк (например, JUnit, pytest, Mocha и др.);

– базовая структура проекта, в которую будет внедрено пользовательское решение.

Плагин также задаёт команды для компиляции и запуска решений, а также пути к исходным файлам и отчётам внутри контейнера. Это позволяет гибко адаптировать среду выполнения под особенности конкретного языка или фреймворка.

2.3 Фронтенд: динамическая подгрузка языков

Плагинная архитектура распространяется и на клиентскую часть приложения. Поскольку пользовательский интерфейс включает редактор кода (на базе Monaco Editor) и селектор доступных языков, фронтенд также должен поддерживать динамическую загрузку языковых модулей.

Список доступных языков и их метаданные (включая ссылку на JavaScript-модуль) предоставляются фронтенду через REST-запрос к веб-серверу. Полученные данные содержат, в частности, URL к ES-модулю [7], реализующему интеграцию с Monaco Editor.

Загрузка модуля осуществляется с использованием динамического импорта ES-модулей. После загрузки модуль регистрирует соответствующий язык в редакторе, обеспечивая подсветку синтаксиса, автодополнение и другие IDE-функции. Модуль может быть размещён в любой директории, доступной через веб-сервер (например, через Nginx), либо обслуживаться непосредственно через HTTP-эндпоинт, реализованный в плагине LanguageWeb.

Такой подход позволяет полностью декларативно расширять список поддерживаемых языков как на сервере, так и на клиенте, сохраняя согласованность между двумя частями системы.

В этой главе были рассмотрены этапы реализации плагинный архитектуры. Пользователю для добавления нового языка требуется написать Java/Kotlin плагин для бэкенда, JS модуль для поддержки редактором языка

(или использовать готовый), а также написать Dockerfile для образа запускаемого языка.

3 Проектирование хранилища MinIO

Глава 3 содержит информацию о проектировании способа хранения файлов в файловом хранилище MinIO, взамен хранению в PostgreSQL.

3.1 Обоснование выбора MinIO

В ходе проектирования архитектуры платформы для решения алгоритмических задач был пересмотрен подход к хранению файловых данных. Изначально предполагалось использовать реляционную СУБД PostgreSQL для хранения как метаданных, так и содержимого файлов (в виде BYTEA или TEXT). Однако такой подход оказался неоптимальным с точки зрения масштабируемости, производительности и разделения ответственности: хранение больших объёмов текстовых данных в реляционной базе усложняет резервное копирование, увеличивает нагрузку на транзакционный движок и затрудняет горизонтальное масштабирование.

В качестве альтернативы было решено внедрить объектное хранилище MinIO — высокопроизводительную, S3-совместимую систему, разработанную специально для хранения неструктурированных данных. Выбор MinIO обусловлен следующими факторами:

– S3-совместимость: MinIO реализует стандартный API Amazon S3 [8], что обеспечивает совместимость с широким спектром инструментов, библиотек и практик, уже устоявшихся в экосистеме облачных приложений;

– простота развёртывания и управления: MinIO может быть легко развёрнут как в локальной среде, так и в распределённом кластере, не требуя сложной инфраструктуры;

– производительность и надёжность: MinIO оптимизирован для высокой пропускной способности и обеспечивает целостность данных за счёт механизмов контроля ошибок и репликации;

– семантическая адекватность: хранение исходного кода, логов и отчётов о тестировании логично рассматривать как набор объектов с чёткой иерархией, а не как записи в реляционной таблице.

Таким образом, MinIO был выбран в качестве основного хранилища для всех файловых артефактов, связанных с пользовательскими решениями и системными задачами, в то время как PostgreSQL продолжает использоваться исключительно для хранения структурированных метаданных (идентификаторы, статусы, временные метки и т.п.).

3.2 Структура объектного хранилища

Хранилище организовано в виде иерархической структуры «bucket/path», где каждый путь отражает семантическую принадлежность файла. Для удобства навигации и изоляции данных принято следующее соглашение об именовании. Пользовательские решения размещаются в пространстве, изолированном по идентификатору пользователя /users/{userId}/solutions/{solutionId}:

– solution.txt - исходный код решения;

– logs.txt - логи выполнения (stdout/stderr);

– result.json - структурированный результат тестирования.

Файл solution.txt содержит именно текстовое представление решения, а не исполняемый код или артефакты компиляции. Такой подход подчёркивает универсальность формата: решение может быть написано на любом языке, но всегда остаётся человекочитаемым текстом. Расширение .txt выбрано намеренно — оно избегает привязки к конкретному языку программирования и подчёркивает, что содержимое предназначено прежде всего для хранения и последующего анализа, а не для прямого исполнения.

Системные задачи (проблемы) хранятся в отдельном пространстве имён, недоступном для модификации пользователями /system/problems/{problemId}/:

– test.txt - тестовые данные или скрипты;

– pre.txt - предварительный код (boilerplate);

– solution-template.txt - шаблон решения для пользователя;

– solution.txt - эталонное (правильное) решение.

Эта структура позволяет легко извлекать все необходимые артефакты задачи при запуске тестирования, а также поддерживать версионность и аудит изменений через внешние механизмы (например, GitOps для загрузки задач).

3.3 Интеграция в архитектуру микросервисов

Внедрение MinIO повлекло за собой модификацию взаимодействия между микросервисами платформы. Обновлённая схема выглядит следующим образом:

1. Web-server:

– принимает от фронтенда запрос на отправку решения;

– сохраняет метаданные решения в PostgreSQL;

– загружает исходный код (solution.txt) в MinIO по пути /users/{userId}/solutions/{solutionId}/solution.txt;

– публикует сообщение в Kafka с указанием solutionId и путей к файлам в MinIO.

2. Test-runner:

– получает сообщение из Kafka;

– загружает из MinIO пользовательское решение (solution.txt), соответствующие файлы задачи (/system/problems/{problemId}/...);

– выполняет тестирование в изолированном контейнере;

– сохраняет результаты (logs.txt, result.json) обратно в MinIO по пути решения;

– публикует сообщение в Kafka с результатами проверки.

Этапы обработки запроса на проверку решения изображены на рисунке 1.

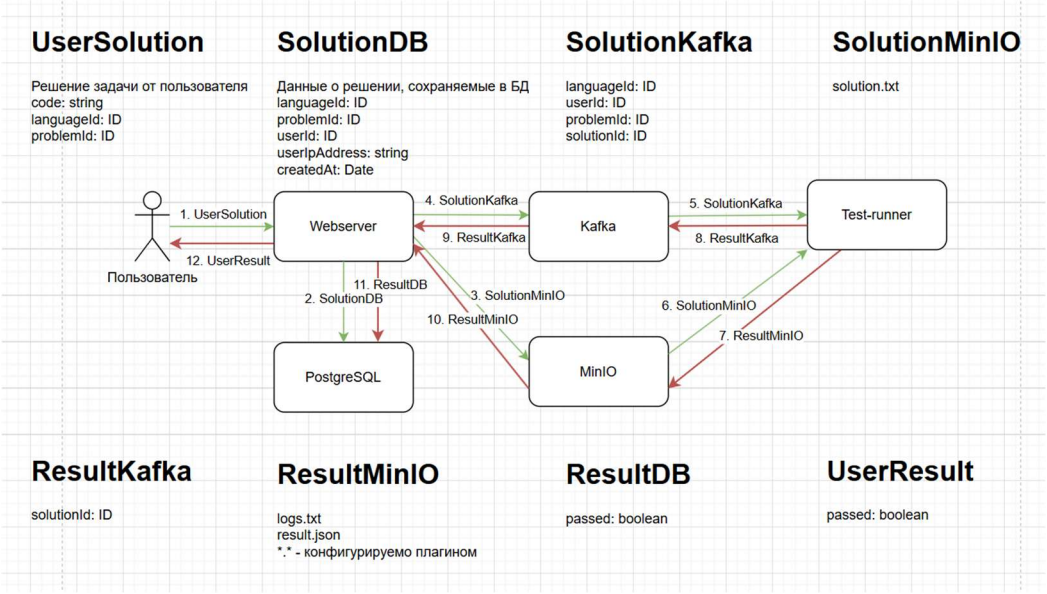

Рисунок 1 – путь по обработке запроса на проверку решения

Таким образом, MinIO становится центральным хранилищем файловых артефактов, обеспечивая единообразный доступ ко всем данным как для операций проверки решений, так и для интерактивного выполнения кода. При этом PostgreSQL остаётся «источником истины» для метаданных, что позволяет сохранить ACID-гарантии для бизнес-логики, не перегружая её файловыми операциями.

Переход от хранения файлов в PostgreSQL к использованию MinIO позволил достичь чёткого разделения ответственности между структурированными и неструктурированными данными, повысить масштабируемость системы и упростить управление жизненным циклом файловых артефактов. Архитектурное решение соответствует современным практикам построения облачно-ориентированных приложений и создаёт прочную основу для дальнейшего развития платформы.

ЗАКЛЮЧЕНИЕ

На основе проведённого анализа обоснована необходимость разработки собственной платформы «SupremeCode», лишенной выявленных недостатков и ориентированной на гибкость, масштабируемость и безопасность.

В ходе выполнения работы были успешно реализованы два ключевых архитектурных компонента:

– спроектирована и описана плагинная архитектура, обеспечивающая расширяемую поддержку произвольных языков программирования как на стороне бэкенда (с использованием ServiceLoader и изолированных classloader’ов), так и на фронтенде (через динамическую загрузку ES-модулей и интеграцию с Monaco Editor);

– разработана система хранения файловых артефактов на базе MinIO, заменившая неэффективное хранение файлов в PostgreSQL и обеспечившая семантически прозрачную, производительную и S3-совместимую модель работы с исходным кодом, логами и результатами тестирования.

Оба решения органично вписываются в общую микросервисную архитектуру платформы, взаимодействуя через Kafka и REST-интерфейсы, и создают прочную основу для дальнейшего развития системы. Плагинный подход позволяет легко интегрировать новые языки и среды выполнения без перекомпиляции ядра приложения, а использование объектного хранилища MinIO обеспечивает надёжность, масштабируемость и соответствие современным DevOps-практикам. В совокупности эти компоненты формируют гибкую, безопасную и легко поддерживаемую экосистему, способную эффективно обслуживать как учебные, так и промышленные сценарии использования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. MinIO — объектное хранилище с открытым исходным кодом [Электронный ресурс], URL: https://min.io/ (дата обращения: 19.01.2026).

2. Плагинная архитектура — статья в Википедии [Электронный ресурс], URL: https://ru.wikipedia.org/wiki/Плагинная_архитектура (дата обращения: 19.01.2026).

3. Java Service Loader — документация Oracle [Электронный ресурс], URL: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/ServiceLoader.html (дата обращения: 19.01.2026).

4. Monaco Editor — редактор кода от Microsoft [Электронный ресурс], URL: https://microsoft.github.io/monaco-editor/ (дата обращения: 19.01.2026).

5. SDK — статья в Википедии [Электронный ресурс], URL: https://ru.wikipedia.org/wiki/SDK (дата обращения: 19.01.2026).

6. URLClassLoader — документация Oracle [Электронный ресурс], URL: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/net/URLClassLoader.html (дата обращения: 19.01.2026).

7. ES modules — документация MDN Web Docs [Электронный ресурс], URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules (дата обращения: 19.01.2026).

8. API Amazon S3 — официальная документация AWS [Электронный ресурс], URL: https://docs.aws.amazon.com/AmazonS3/latest/API/Welcome.html (дата обращения: 19.01.2026).